MIMIC-IV PostgreSQL Build Pipeline

Orchestrates the full MIMIC-IV + MIMIC-IV-ED PostgreSQL setup using plain Docker containers.
Each step checks whether it has already been completed and skips it on subsequent runs.

All configuration is read from a `.env` file — see `.env.local` for the template.

MIMIC-IV core

Step	Description
1	Verify Docker is running
2	Ensure Docker network exists
3	Build PostgreSQL image (`docker/Dockerfile.db`)
4	Start PostgreSQL container
5	Wait for PostgreSQL to be ready
6	Create schemas and tables (`mimic-iv create.sql`)
7	Load MIMIC-IV data (`mimic-iv load_gz.sql`)
8	Add primary key constraints (`mimic-iv constraint.sql`)
9	Create indexes (`mimic-iv index.sql`)
10	Validate row counts (`mimic-iv validate.sql`)

MIMIC-IV-ED — data loaded from `MIMIC_DATA_DIR/ed/`

Step	Description
11	Create `mimiciv_ed` schema and tables (`mimic-iv-ed create.sql`)
12	Load MIMIC-IV-ED data (`mimic-iv-ed load_gz.sql`)
13	Add ED primary/foreign key constraints (`mimic-iv-ed constraint.sql`)
14	Create ED indexes (`mimic-iv-ed index.sql`)
15	Validate ED row counts (`mimic-iv-ed validate.sql`)

> **Run all steps**: *Kernel → Restart & Run All*
> **Run one step**: call the individual `stepN_*()` function in a new

> cell.

Imports & Logging

In [1]:
```
import json
import logging
import os
import re
import subprocess
import tempfile
import time
from datetime import datetime
from pathlib import Path

import psycopg
from dotenv import load_dotenv

LOG_FORMAT = "%(asctime)s [%(levelname)-8s] %(message)s"
LOG_DATE   = "%Y-%m-%d %H:%M:%S"

logging.basicConfig(
    level=logging.INFO,
    format=LOG_FORMAT,
    datefmt=LOG_DATE,
    handlers=[
        logging.StreamHandler(),
        logging.FileHandler("build_mimic.log", encoding="utf-8"),
    ],
)
log = logging.getLogger(__name__)
```

Configuration

Values are loaded from `.env`. Edit that file to change any setting.

In [2]:
```
load_dotenv()

POSTGRES_DB        = os.environ.get("POSTGRES_DB",        "mimiciv")
POSTGRES_USER      = os.environ.get("POSTGRES_USER",      "mimicuser")
POSTGRES_PASSWORD  = os.environ.get("POSTGRES_PASSWORD",  "mimicpass")
POSTGRES_PORT      = int(os.environ.get("POSTGRES_PORT",  "5432"))
DB_CONTAINER_NAME  = os.environ.get("DB_CONTAINER_NAME",  "mimic_postg
DB_IMAGE_NAME      = os.environ.get("DB_IMAGE_NAME",      "mimic-db")
DOCKER_NETWORK     = os.environ.get("DOCKER_NETWORK",     "mimic-net")
MIMIC_DATA_DIR     = os.environ.get("MIMIC_DATA_DIR",     "")
MIMIC_CODE_DIR     = os.environ.get("MIMIC_CODE_DIR",     "./mimic-cod
LOAD_LIMITS_FILE   = os.environ.get("LOAD_LIMITS_FILE",   "")
```

```
MIMIC_BUILD_DIR      = Path(MIMIC_CODE_DIR) / "mimic-iv"    / "buildmi
MIMIC_ED_BUILD_DIR   = Path(MIMIC_CODE_DIR) / "mimic-iv-ed" / "buildmi
MIMIC_CONCEPTS_DIR   = Path(MIMIC_CODE_DIR) / "mimic-iv"    / "concept

print(f"Database      : {POSTGRES_DB}")
print(f"User          : {POSTGRES_USER}")
print(f"Container     : {DB_CONTAINER_NAME}")
print(f"Network       : {DOCKER_NETWORK}")
print(f"Port          : {POSTGRES_PORT}")
print(f"Data dir      : {MIMIC_DATA_DIR or '(not set)'}")
print(f"Limits file   : {LOAD_LIMITS_FILE or '(not set — no row limits
print(f"mimic-code    : {Path(MIMIC_CODE_DIR).resolve()}")
print(f"Build dir     : {MIMIC_BUILD_DIR.resolve()}")
print(f"ED build dir  : {MIMIC_ED_BUILD_DIR.resolve()}")
print(f"Concepts dir  : {MIMIC_CONCEPTS_DIR.resolve()}")
```

```
Database      : mimiciv
User          : mimicuser
Container     : mimic_postgres
Network       : mimic-net
Port          : 5432
Data dir      : /Volumes/ExternalSSD/AI_For_Healthcare_files/data
Limits file   : ./limits.json
mimic-code    : /Users/mfanti/Masters_UniversityOfTexas_Austin/AI_for_H
ealthCare/LearningProject2/mimic-code
Build dir     : /Users/mfanti/Masters_UniversityOfTexas_Austin/AI_for_H
ealthCare/LearningProject2/mimic-code/mimic-iv/buildmimic/postgres
ED build dir  : /Users/mfanti/Masters_UniversityOfTexas_Austin/AI_for_H
ealthCare/LearningProject2/mimic-code/mimic-iv-ed/buildmimic/postgres
Concepts dir  : /Users/mfanti/Masters_UniversityOfTexas_Austin/AI_for_H
ealthCare/LearningProject2/mimic-code/mimic-iv/concepts_postgres
```

Core Helpers

In [3]:

```
def run(cmd: list[str], *, check: bool = True, capture: bool = False)
    """Run a subprocess command with logging. Returns CompletedProcess
    log.debug("Executing: %s", " ".join(str(c) for c in cmd))
    return subprocess.run(cmd, check=check, capture_output=capture, te


def db_connect() -> psycopg.Connection:
    """Open a psycopg connection from the host to the running PostgreS
    return psycopg.connect(
        host="localhost",
        port=POSTGRES_PORT,
        user=POSTGRES_USER,
        password=POSTGRES_PASSWORD,
        dbname=POSTGRES_DB,
    )
```

```python
def run_psql_in_docker(
    script_path: Path,
    extra_psql_vars: dict[str, str] | None = None,
    extra_volumes: list[str] | None = None,
    capture: bool = False,
) -> subprocess.CompletedProcess:
    """
    Run a psql script inside a temporary postgres:17 container on DOCK
    connecting to DB_CONTAINER_NAME by container name.
    """
    abs_script = Path(script_path).resolve()
    conn_str = (
        f"host={DB_CONTAINER_NAME} "
        f"dbname={POSTGRES_DB} "
        f"user={POSTGRES_USER} "
        f"password={POSTGRES_PASSWORD}"
    )

    cmd = [
        "docker", "run", "--rm",
        "--network", DOCKER_NETWORK,
        "-v", f"{abs_script}:/script.sql:ro",
    ]
    if extra_volumes:
        for vol in extra_volumes:
            cmd += ["-v", vol]

    cmd += ["postgres:17", "psql", conn_str, "-v", "ON_ERROR_STOP=1"]
    if extra_psql_vars:
        for key, val in extra_psql_vars.items():
            cmd += ["-v", f"{key}={val}"]

    cmd += ["-f", "/script.sql"]
    return run(cmd, capture=capture)
```

Row-Limit Helpers

When `LOAD_LIMITS_FILE` is set, the loader caps each table's `\COPY` stream with `head -n N`
and adds FK constraints as `NOT VALID` to avoid referential integrity errors on truncated data.

In [4]:
```python
def load_limits() -> dict | None:
    """
    Load per-table row limits from LOAD_LIMITS_FILE.
    Returns the parsed dict, or None if LOAD_LIMITS_FILE is unset/miss
    Expected JSON shape: { "default": 10000, "overrides": { "patients"
    """
    if not LOAD_LIMITS_FILE:
        return None
```

```
    path = Path(LOAD_LIMITS_FILE)
    if not path.exists():
        log.warning("LOAD_LIMITS_FILE '%s' not found — loading without
        return None
    with path.open() as f:
        return json.load(f)


def _row_limit(table: str, limits: dict) -> int | None:
    """Return the row limit for a table, or None for unlimited."""
    override = limits.get("overrides", {}).get(table)
    if override is not None:
        return override
    return limits.get("default")


# MIMIC-IV core: schema-qualified names, e.g. \COPY mimiciv_hosp.patie
_COPY_RE = re.compile(
    r"(\\COPY\s+\w+\.(\w+)\s+FROM\s+PROGRAM\s+')(gzip -dc [^']+)('.*)"
    re.IGNORECASE,
)

# MIMIC-IV-ED: unqualified names (relies on SET search_path TO mimiciv
_ED_COPY_RE = re.compile(
    r"(\\COPY\s+(\w+)\s+FROM\s+PROGRAM\s+')(gzip -dc [^']+)('.*)",
    re.IGNORECASE,
)

_TIMING_RE = re.compile(r"\[TABLE_(START|END)\] (\w+) (\d{4}-\d{2}-\d{

_TS_FMT = "YYYY-MM-DD HH24:MI:SS"


def _find_core_table_paths(original_sql: Path, data_path: Path) -> dic
    """
    MIMIC-IV v1 stored admissions/patients/transfers under core/; v2 m
    For any \\COPY whose CSV is missing from hosp/ but present in core
    of table_name -> absolute container path (under /data/mimic/core/)
    patched to use the correct location.
    """
    hosp_dir = data_path / "hosp"
    core_dir = data_path / "core"
    core_tables: dict[str, str] = {}

    if not core_dir.is_dir():
        return core_tables

    for line in original_sql.read_text().splitlines():
        m = _COPY_RE.match(line)
        if not m:
            continue
        _, table, program, _ = m.groups()
```

```
        filename = program.split()[-1]
        if not (hosp_dir / filename).exists() and (core_dir / filename
            core_tables[table] = f"/data/mimic/core/{filename}"
            log.info("  %-30s → sourcing from core/ (not in hosp/)", t

    return core_tables


def _truncate_all_tables(
    schemas: tuple[str, ...] = ("mimiciv_hosp", "mimiciv_icu"),
) -> None:
    """
    Truncate all data tables in the given schemas before a reload.
    Defaults to mimiciv_hosp + mimiciv_icu; pass schemas=('mimiciv_ed'
    """
    with db_connect() as conn:
        cur = conn.cursor()
        cur.execute(
            """
            SELECT table_schema, table_name
            FROM information_schema.tables
            WHERE table_schema = ANY(%s)
              AND table_type = 'BASE TABLE'
            ORDER BY table_schema, table_name
            """,
            (list(schemas),),
        )
        tables = cur.fetchall()
        if not tables:
            return
        for schema, table in tables:
            cur.execute(f"TRUNCATE TABLE {schema}.{table}")
        conn.commit()
        log.info("Truncated %d tables for clean reload.", len(tables))


def _drop_fk_constraints(
    schemas: tuple[str, ...] = ("mimiciv_hosp", "mimiciv_icu"),
) -> None:
    """
    Drop all FK constraints in the given schemas before a data load.
    Defaults to mimiciv_hosp + mimiciv_icu; pass schemas=('mimiciv_ed'
    """
    with db_connect() as conn:
        cur = conn.cursor()
        cur.execute(
            """
            SELECT c.conname, n.nspname, cl.relname
            FROM pg_constraint c
            JOIN pg_class cl ON cl.oid = c.conrelid
            JOIN pg_namespace n ON n.oid = cl.relnamespace
            WHERE c.contype = 'f'
```

```
              AND n.nspname = ANY(%s)
            ORDER BY n.nspname, cl.relname, c.conname
            """,
            (list(schemas),),
        )
        fk_constraints = cur.fetchall()

        if not fk_constraints:
            return

        log.info(
            "Dropping %d FK constraint(s) before load (constraint.sql
            len(fk_constraints),
        )
        for conname, nspname, relname in fk_constraints:
            cur.execute(
                f"ALTER TABLE {nspname}.{relname} DROP CONSTRAINT IF E
            )
            log.info("  Dropped FK: %s.%s -> %s", nspname, relname, co
        conn.commit()


def generate_timed_load_sql(
    original_sql: Path,
    out_dir: Path,
    limits: dict | None = None,
    core_tables: dict[str, str] | None = None,
) -> Path:
    """
    Rewrite load_gz.sql to wrap each \\COPY with SELECT clock_timestam
    that emit [TABLE_START] / [TABLE_END] markers for per-table load t
    When limits is provided, also pipes each gzip stream through head
    When core_tables is provided, overrides the file path for tables f
    """
    lines = []
    for line in original_sql.read_text().splitlines(keepends=True):
        m = _COPY_RE.match(line.rstrip("\n"))
        if m:
            prefix, table, program, suffix = m.groups()
            if core_tables and table in core_tables:
                program = f"gzip -dc {core_tables[table]}"
            if limits is not None:
                limit = _row_limit(table, limits)
                if limit is not None:
                    program = f"{program} | head -n {limit + 1}"
                    log.info("  %-30s → limit %d rows", table, limit)
            lines.append(
                f"SELECT '[TABLE_START] {table} ' || to_char(clock_tim
            )
            lines.append(f"{prefix}{program}{suffix}\n")
            lines.append(
                f"SELECT '[TABLE_END] {table} ' || to_char(clock_times
```

```
            )
        else:
            lines.append(line)

    out_path = out_dir / "load_timed.sql"
    out_path.write_text("".join(lines))
    return out_path


def generate_timed_ed_load_sql(
    original_sql: Path,
    out_dir: Path,
    limits: dict | None = None,
) -> Path:
    """
    Rewrite the ED load_gz.sql to wrap each \\COPY with timing markers
    Uses _ED_COPY_RE because the ED script uses unqualified table name
    (SET search_path TO mimiciv_ed). No core-table path fallback neede
    """
    lines = []
    for line in original_sql.read_text().splitlines(keepends=True):
        m = _ED_COPY_RE.match(line.rstrip("\n"))
        if m:
            prefix, table, program, suffix = m.groups()
            if limits is not None:
                limit = _row_limit(table, limits)
                if limit is not None:
                    program = f"{program} | head -n {limit + 1}"
                    log.info("  %-30s → limit %d rows", table, limit)
            lines.append(
                f"SELECT '[TABLE_START] {table} ' || to_char(clock_tim
            )
            lines.append(f"{prefix}{program}{suffix}\n")
            lines.append(
                f"SELECT '[TABLE_END] {table} ' || to_char(clock_times
            )
        else:
            lines.append(line)

    out_path = out_dir / "load_ed_timed.sql"
    out_path.write_text("".join(lines))
    return out_path


def _log_table_timings(stdout: str) -> None:
    """Parse [TABLE_START]/[TABLE_END] markers from psql output and lo
    starts: dict[str, datetime] = {}
    for line in stdout.splitlines():
        m = _TIMING_RE.search(line)
        if not m:
            continue
        kind, table, ts = m.groups()
```

```
        try:
            dt = datetime.fromisoformat(ts)
        except ValueError:
            continue
        if kind == "START":
            starts[table] = dt
            log.info("  %-30s  started  %s", table, ts)
        elif kind == "END" and table in starts:
            secs = (dt - starts[table]).total_seconds()
            log.info("  %-30s  finished %s  (%.1f s)", table, ts, secs


_REFERENCES_RE = re.compile(
    r"(REFERENCES\s+\S+\s*\([^)]+\))\s*;",
    re.IGNORECASE,
)


def generate_not_valid_constraint_sql(original_sql: Path, out_dir: Pat
    """
    Rewrite constraint.sql so every FOREIGN KEY is added with NOT VALI
    meaning PostgreSQL won't scan existing rows for violations. Correc
    a truncated/limited dataset where referential integrity isn't guar
    """
    text = original_sql.read_text()
    patched = _REFERENCES_RE.sub(r"\1 NOT VALID;", text)
    out_path = out_dir / "constraint_not_valid.sql"
    out_path.write_text(patched)
    return out_path
```

Step 1 — Verify Docker is Running

In [5]:
```
def step1_check_docker() -> None:
    log.info("=" * 60)
    log.info("STEP 1: Verifying Docker is running")
    log.info("=" * 60)

    result = run(["docker", "info"], check=False, capture=True)
    if result.returncode != 0:
        raise RuntimeError("Docker is not running or not accessible. S

    version_result = run(["docker", "version", "--format", "{{.Server.
                         check=False, capture=True)
    version = version_result.stdout.strip() if version_result.returnco
    log.info("Docker is running. Server version: %s", version)
```

Step 2 — Ensure Docker Network Exists

In [6]:
```
def step2_ensure_network() -> None:
```

```
    log.info("=" * 60)
    log.info("STEP 2: Ensuring Docker network '%s' exists", DOCKER_NET
    log.info("=" * 60)

    result = run(["docker", "network", "inspect", DOCKER_NETWORK], che
    if result.returncode == 0:
        log.info("SKIP: Network '%s' already exists.", DOCKER_NETWORK)
        return

    log.info("Network '%s' not found — creating it ...", DOCKER_NETWOR
    run(["docker", "network", "create", DOCKER_NETWORK])
    log.info("Network '%s' created successfully.", DOCKER_NETWORK)
```

Step 3 — Build PostgreSQL Image

In [7]:
```
def step3_build_image() -> None:
    log.info("=" * 60)
    log.info("STEP 3: Building PostgreSQL image '%s'", DB_IMAGE_NAME)
    log.info("=" * 60)

    result = run(["docker", "image", "inspect", DB_IMAGE_NAME], check=
    if result.returncode == 0:
        log.info("SKIP: Image '%s' already exists — skipping build.",
        log.info("      To force a rebuild: docker rmi %s", DB_IMAGE_N
        return

    dockerfile = Path("docker/Dockerfile.db")
    if not dockerfile.exists():
        raise RuntimeError(f"Dockerfile not found at '{dockerfile}'. R

    log.info("Building image '%s' from %s ...", DB_IMAGE_NAME, dockerf
    run(["docker", "build", "-f", str(dockerfile), "-t", DB_IMAGE_NAME
    log.info("Image '%s' built successfully.", DB_IMAGE_NAME)
```

Step 4 — Start PostgreSQL Container

In [8]:
```
def _ensure_container_on_network() -> None:
    """Connect DB_CONTAINER_NAME to DOCKER_NETWORK if it isn't already
    inspect = run(
        ["docker", "container", "inspect", DB_CONTAINER_NAME],
        check=False, capture=True,
    )
    if inspect.returncode != 0:
        return
    info = json.loads(inspect.stdout)
    networks = info[0].get("NetworkSettings", {}).get("Networks", {})
    if DOCKER_NETWORK not in networks:
        log.warning(
            "Container '%s' is not on network '%s' — connecting it now
```

```
            DB_CONTAINER_NAME, DOCKER_NETWORK,
        )
        run(["docker", "network", "connect", DOCKER_NETWORK, DB_CONTAI
        log.info("Container '%s' connected to network '%s'.", DB_CONTA
    else:
        log.info("Container '%s' is already on network '%s'.", DB_CONT


def step4_start_db() -> None:
    log.info("=" * 60)
    log.info("STEP 4: Starting PostgreSQL container '%s'", DB_CONTAINE
    log.info("=" * 60)

    inspect = run(
        ["docker", "container", "inspect", DB_CONTAINER_NAME],
        check=False, capture=True,
    )

    if inspect.returncode == 0:
        info  = json.loads(inspect.stdout)
        state = info[0]["State"]["Status"]

        if state == "running":
            log.info("Container '%s' is already running.", DB_CONTAINE
        else:
            log.info("Container '%s' exists but is in state '%s' — sta
                     DB_CONTAINER_NAME, state)
            run(["docker", "start", DB_CONTAINER_NAME])
            log.info("Container '%s' started.", DB_CONTAINER_NAME)

        _ensure_container_on_network()
        return

    data_dir = Path("data/db").resolve()
    data_dir.mkdir(parents=True, exist_ok=True)
    log.info("Data directory: %s", data_dir)

    log.info("Creating and starting container '%s' ...", DB_CONTAINER_
    run([
        "docker", "run", "-d",
        "--name",      DB_CONTAINER_NAME,
        "--network",   DOCKER_NETWORK,
        "--shm-size",  "256m",
        "-p",          f"{POSTGRES_PORT}:5432",
        "-v",          f"{data_dir}:/var/lib/postgresql/data",
        DB_IMAGE_NAME,
    ])
    log.info("Container '%s' created and started.", DB_CONTAINER_NAME)
```

Step 5 — Wait for PostgreSQL to be Ready

In [9]:
```
def step5_wait_for_db(max_attempts: int = 30, delay: float = 5.0) -> N
    log.info("=" * 60)
    log.info("STEP 5: Waiting for PostgreSQL to accept connections")
    log.info("=" * 60)
    log.info("Polling every %.0f s (up to %d attempts) ...", delay, ma

    for attempt in range(1, max_attempts + 1):
        try:
            conn = db_connect()
            conn.close()
            log.info("PostgreSQL is ready (attempt %d/%d).", attempt,
            return
        except Exception as exc:
            log.info(
                "Not ready yet [%d/%d]: %s — retrying in %.0f s ...",
                attempt, max_attempts, exc, delay,
            )
            time.sleep(delay)

    raise RuntimeError(
        f"PostgreSQL did not become ready after {max_attempts} attempt
        f"Check logs with: docker logs {DB_CONTAINER_NAME}"
    )
```

Step 6 — Create MIMIC-IV Schemas and Tables

In [10]:
```
def step6_create_schema() -> None:
    log.info("=" * 60)
    log.info("STEP 6: Creating MIMIC-IV schemas and tables (create.sql
    log.info("=" * 60)

    with db_connect() as conn:
        cur = conn.cursor()
        cur.execute("""
            SELECT COUNT(*)
            FROM information_schema.schemata
            WHERE schema_name IN ('mimiciv_hosp', 'mimiciv_icu', 'mimi
        """)
        schema_count = cur.fetchone()[0]

    if schema_count == 3:
        with db_connect() as conn:
            cur = conn.cursor()
            cur.execute("""
                SELECT COUNT(*)
                FROM information_schema.tables
                WHERE table_schema = 'mimiciv_hosp'
            """)
            table_count = cur.fetchone()[0]
```

```
        if table_count > 0:
            log.info(
                "SKIP: All 3 MIMIC-IV schemas already exist with %d ta
                table_count,
            )
            return
        log.info("Schemas exist but have no tables — re-running create
    else:
        log.info("Found %d/3 required schemas — running create.sql ...

    script = MIMIC_BUILD_DIR / "create.sql"
    if not script.exists():
        raise RuntimeError(
            f"Script not found: {script}. "
            f"Ensure mimic-code is cloned at MIMIC_CODE_DIR='{MIMIC_CO
        )

    log.warning("NOTE: create.sql drops and recreates schemas. Existin
    run_psql_in_docker(script)
    log.info("Schemas and tables created successfully.")
```

Step 7 — Load MIMIC-IV Data

> ⚠️ This step can take **many hours** for the full dataset.
> If `LOAD_LIMITS_FILE` is set, row limits are applied per table via `head -n N`.

In [11]:
```
def step7_load_data() -> None:
    log.info("=" * 60)
    log.info("STEP 7: Loading MIMIC-IV data (load_gz.sql)")
    log.info("=" * 60)
    log.info("NOTE: This step can take many hours for the full dataset

    if not MIMIC_DATA_DIR:
        raise RuntimeError("MIMIC_DATA_DIR is not set in .env")

    data_path = Path(MIMIC_DATA_DIR).resolve()
    if not data_path.exists():
        raise RuntimeError(f"MIMIC_DATA_DIR '{data_path}' does not exi

    for subdir in ("hosp", "icu"):
        if not (data_path / subdir).is_dir():
            raise RuntimeError(f"Expected subdirectory '{subdir}/' not

    try:
        with db_connect() as conn:
            cur = conn.cursor()
            cur.execute("SELECT COUNT(*) FROM mimiciv_hosp.patients")
            row_count = cur.fetchone()[0]
```

```
    except Exception:
        row_count = 0

    if row_count > 0:
        log.info(
            "SKIP: mimiciv_hosp.patients already contains %d rows — da
            row_count,
        )
        return

    log.info("No data found in mimiciv_hosp.patients — starting data l
    _drop_fk_constraints()
    _truncate_all_tables()
    log.info("Data directory: %s", data_path)
    log.info("This may take many hours. Per-table timing will appear b

    script = MIMIC_BUILD_DIR / "load_gz.sql"
    if not script.exists():
        raise RuntimeError(f"Script not found: {script}")

    limits = load_limits()
    if limits is not None:
        log.info("Row limits active (from '%s'): default=%s", LOAD_LIM

    core_tables = _find_core_table_paths(script, data_path)
    if core_tables:
        log.info(
            "Sourcing %d table(s) from core/ instead of hosp/: %s",
            len(core_tables), ", ".join(core_tables),
        )

    with tempfile.TemporaryDirectory() as tmp:
        timed_script = generate_timed_load_sql(script, Path(tmp), limi
        result = run_psql_in_docker(
            timed_script,
            extra_psql_vars={"mimic_data_dir": "/data/mimic"},
            extra_volumes=[f"{data_path}:/data/mimic:ro"],
            capture=True,
        )

    _log_table_timings(result.stdout)
    log.info("Data loaded successfully.")
```

Step 8 — Add Primary Key & Foreign Key Constraints

When row limits are active, FK constraints are added as `NOT VALID` so PostgreSQL
skips the data scan (correct for a truncated dataset).

In [12]:
```
def step8_add_constraints() -> None:
    log.info("=" * 60)
    log.info("STEP 8: Adding primary key constraints (constraint.sql)"
    log.info("=" * 60)

    with db_connect() as conn:
        cur = conn.cursor()
        cur.execute("""
            SELECT COUNT(*)
            FROM pg_constraint c
            JOIN pg_namespace n ON n.oid = c.connamespace
            WHERE n.nspname = 'mimiciv_hosp'
              AND c.conname = 'admissions_pk'
        """)
        pk_exists = cur.fetchone()[0]
        cur.execute("""
            SELECT COUNT(*)
            FROM pg_constraint c
            JOIN pg_namespace n ON n.oid = c.connamespace
            WHERE n.nspname = 'mimiciv_hosp'
              AND c.conname = 'admissions_patients_fk'
        """)
        fk_exists = cur.fetchone()[0]

    if pk_exists and fk_exists:
        log.info("SKIP: Constraints already exist (admissions_pk + adm
        return

    script = MIMIC_BUILD_DIR / "constraint.sql"
    if not script.exists():
        raise RuntimeError(f"Script not found: {script}")

    if load_limits() is not None:
        log.info("Row limits active — adding FK constraints with NOT V
        with tempfile.TemporaryDirectory() as tmp:
            patched_script = generate_not_valid_constraint_sql(script,
            run_psql_in_docker(patched_script)
    else:
        log.info("Constraints not found — running constraint.sql ...")
        run_psql_in_docker(script)
    log.info("Constraints added successfully.")
```

Step 9 — Create Indexes

In [13]:
```
def step9_create_indexes() -> None:
    log.info("=" * 60)
    log.info("STEP 9: Creating indexes (index.sql)")
    log.info("=" * 60)

    with db_connect() as conn:
        cur = conn.cursor()
```

```
        cur.execute("""
            SELECT COUNT(*)
            FROM pg_indexes
            WHERE schemaname = 'mimiciv_hosp'
              AND indexname = 'admissions_idx01'
        """)
        exists = cur.fetchone()[0]

    if exists:
        log.info("SKIP: Index 'admissions_idx01' already exists — skip
        return

    script = MIMIC_BUILD_DIR / "index.sql"
    if not script.exists():
        raise RuntimeError(f"Script not found: {script}")

    log.info("Indexes not found — running index.sql ...")
    run_psql_in_docker(script)
    log.info("Indexes created successfully.")
```

Step 10 — Validate Row Counts

In [14]:
```
def step10_validate() -> None:
    log.info("=" * 60)
    log.info("STEP 10: Validating row counts (validate.sql)")
    log.info("=" * 60)

    script = MIMIC_BUILD_DIR / "validate.sql"
    if not script.exists():
        raise RuntimeError(f"Script not found: {script}")

    abs_script = script.resolve()
    conn_str = (
        f"host={DB_CONTAINER_NAME} "
        f"dbname={POSTGRES_DB} "
        f"user={POSTGRES_USER} "
        f"password={POSTGRES_PASSWORD}"
    )

    result = run(
        [
            "docker", "run", "--rm",
            "--network", DOCKER_NETWORK,
            "-v", f"{abs_script}:/script.sql:ro",
            "postgres:17",
            "psql", conn_str,
            "-f", "/script.sql",
        ],
        check=False,
        capture=True,
    )
```

```
    print(result.stdout)
    if result.stderr.strip():
        log.warning("Validation stderr:\n%s", result.stderr)

    if result.returncode != 0:
        log.warning(
            "validate.sql exited with code %d — some row counts may no
            result.returncode,
        )
    else:
        log.info("Validation completed — all row counts match expected
```

MIMIC-IV-ED Steps (11–15)

The following steps build the `mimiciv_ed` schema from the MIMIC-IV-ED dataset.
They require the ED CSV files to be present at `MIMIC_DATA_DIR/ed/`
(see the tutorial for download instructions).

ED schema — 6 tables in `mimiciv_ed`:

Table	Description
`edstays`	ED visit records (one row per visit)
`diagnosis`	ED discharge diagnoses
`triage`	Triage vital signs (one per stay)
`vitalsign`	Ongoing vital sign measurements
`medrecon`	Medication reconciliation at admission
`pyxis`	Automated dispensing cabinet records

Step 11 — Create MIMIC-IV-ED Schema and Tables

In [15]:
```
def step11_create_ed_schema() -> None:
    log.info("=" * 60)
    log.info("STEP 11: Creating MIMIC-IV-ED schema and tables (ED crea
    log.info("=" * 60)

    with db_connect() as conn:
        cur = conn.cursor()
        cur.execute("""
```

```
            SELECT COUNT(*)
            FROM information_schema.schemata
            WHERE schema_name = 'mimiciv_ed'
        """)
        schema_exists = cur.fetchone()[0]

    if schema_exists:
        with db_connect() as conn:
            cur = conn.cursor()
            cur.execute("""
                SELECT COUNT(*)
                FROM information_schema.tables
                WHERE table_schema = 'mimiciv_ed'
            """)
            table_count = cur.fetchone()[0]

        if table_count > 0:
            log.info(
                "SKIP: mimiciv_ed schema already exists with %d tables
                table_count,
            )
            return
        log.info("mimiciv_ed schema exists but has no tables — re-runn
    else:
        log.info("mimiciv_ed schema not found — running ED create.sql

    script = MIMIC_ED_BUILD_DIR / "create.sql"
    if not script.exists():
        raise RuntimeError(
            f"Script not found: {script}. "
            f"Ensure mimic-code is cloned at MIMIC_CODE_DIR='{MIMIC_CO
        )

    log.warning("NOTE: ED create.sql drops and recreates mimiciv_ed. E
    run_psql_in_docker(script)
    log.info("ED schema and tables created successfully.")
```

Step 12 — Load MIMIC-IV-ED Data

> ⚠️ Requires `MIMIC_DATA_DIR/ed/` to exist with the ED `.csv.gz` files.
> If `LOAD_LIMITS_FILE` is set, row limits are applied the same way as for MIMIC-IV core.

In [16]:
```
def step12_load_ed_data() -> None:
    log.info("=" * 60)
    log.info("STEP 12: Loading MIMIC-IV-ED data (ED load_gz.sql)")
    log.info("=" * 60)
```

```
    if not MIMIC_DATA_DIR:
        raise RuntimeError(
            "MIMIC_DATA_DIR is not set in .env. "
            "Set it to the directory whose ed/ subdirectory contains t
        )

    data_path = Path(MIMIC_DATA_DIR).resolve()
    ed_path = data_path / "ed"
    if not ed_path.is_dir():
        raise RuntimeError(
            f"Expected ED subdirectory '{ed_path}' not found. "
            f"Place MIMIC-IV-ED CSV files under {data_path}/ed/"
        )

    # Idempotency check: skip if edstays already has rows
    try:
        with db_connect() as conn:
            cur = conn.cursor()
            cur.execute("SELECT COUNT(*) FROM mimiciv_ed.edstays")
            row_count = cur.fetchone()[0]
    except Exception:
        row_count = 0

    if row_count > 0:
        log.info(
            "SKIP: mimiciv_ed.edstays already contains %d rows — ED da
            row_count,
        )
        return

    log.info("No data found in mimiciv_ed.edstays — starting ED data l
    _drop_fk_constraints(schemas=("mimiciv_ed",))
    _truncate_all_tables(schemas=("mimiciv_ed",))
    log.info("ED data directory: %s", ed_path)

    script = MIMIC_ED_BUILD_DIR / "load_gz.sql"
    if not script.exists():
        raise RuntimeError(f"Script not found: {script}")

    limits = load_limits()
    if limits is not None:
        log.info("Row limits active (from '%s'): default=%s", LOAD_LIM

    with tempfile.TemporaryDirectory() as tmp:
        timed_script = generate_timed_ed_load_sql(script, Path(tmp), l
        result = run_psql_in_docker(
            timed_script,
            # The ED load script uses \cd :mimic_data_dir then relativ
            # Mount the ed/ directory at /data/mimic/ed and point mimi
            extra_psql_vars={"mimic_data_dir": "/data/mimic/ed"},
            extra_volumes=[f"{ed_path}:/data/mimic/ed:ro"],
            capture=True,
```

```
        )

    _log_table_timings(result.stdout)
    log.info("ED data loaded successfully.")
```

Step 13 — Add MIMIC-IV-ED Constraints

When row limits are active, FK constraints are added as `NOT VALID` (same behaviour as step 8).

In [17]:
```
def step13_add_ed_constraints() -> None:
    log.info("=" * 60)
    log.info("STEP 13: Adding MIMIC-IV-ED constraints (ED constraint.s
    log.info("=" * 60)

    with db_connect() as conn:
        cur = conn.cursor()
        cur.execute("""
            SELECT COUNT(*)
            FROM pg_constraint c
            JOIN pg_namespace n ON n.oid = c.connamespace
            WHERE n.nspname = 'mimiciv_ed'
              AND c.conname = 'edstays_pk'
        """)
        pk_exists = cur.fetchone()[0]
        cur.execute("""
            SELECT COUNT(*)
            FROM pg_constraint c
            JOIN pg_namespace n ON n.oid = c.connamespace
            WHERE n.nspname = 'mimiciv_ed'
              AND c.conname = 'diagnosis_edstays_fk'
        """)
        fk_exists = cur.fetchone()[0]

    if pk_exists and fk_exists:
        log.info(
            "SKIP: ED constraints already exist (edstays_pk + diagnosi
        )
        return

    script = MIMIC_ED_BUILD_DIR / "constraint.sql"
    if not script.exists():
        raise RuntimeError(f"Script not found: {script}")

    if load_limits() is not None:
        log.info("Row limits active — adding ED FK constraints with NO
        with tempfile.TemporaryDirectory() as tmp:
            patched_script = generate_not_valid_constraint_sql(script,
            run_psql_in_docker(patched_script)
    else:
```

```
        log.info("ED constraints not found — running ED constraint.sql
        run_psql_in_docker(script)
    log.info("ED constraints added successfully.")
```

Step 14 — Create MIMIC-IV-ED Indexes

In [18]:
```
def step14_create_ed_indexes() -> None:
    log.info("=" * 60)
    log.info("STEP 14: Creating MIMIC-IV-ED indexes (ED index.sql)")
    log.info("=" * 60)

    with db_connect() as conn:
        cur = conn.cursor()
        cur.execute("""
            SELECT COUNT(*)
            FROM pg_indexes
            WHERE schemaname = 'mimiciv_ed'
              AND indexname = 'edstays_idx01'
        """)
        exists = cur.fetchone()[0]

    if exists:
        log.info("SKIP: Index 'edstays_idx01' already exists — skippin
        return

    script = MIMIC_ED_BUILD_DIR / "index.sql"
    if not script.exists():
        raise RuntimeError(f"Script not found: {script}")

    log.info("ED indexes not found — running ED index.sql ...")
    run_psql_in_docker(script)
    log.info("ED indexes created successfully.")
```

Step 15 — Validate MIMIC-IV-ED Row Counts

In [19]:
```
def step15_validate_ed() -> None:
    log.info("=" * 60)
    log.info("STEP 15: Validating MIMIC-IV-ED row counts (ED validate.
    log.info("=" * 60)

    script = MIMIC_ED_BUILD_DIR / "validate.sql"
    if not script.exists():
        raise RuntimeError(f"Script not found: {script}")

    abs_script = script.resolve()
    conn_str = (
        f"host={DB_CONTAINER_NAME} "
        f"dbname={POSTGRES_DB} "
        f"user={POSTGRES_USER} "
```

```
        f"password={POSTGRES_PASSWORD}"
    )

    result = run(
        [
            "docker", "run", "--rm",
            "--network", DOCKER_NETWORK,
            "-v", f"{abs_script}:/script.sql:ro",
            "postgres:17",
            "psql", conn_str,
            "-f", "/script.sql",
        ],
        check=False,
        capture=True,
    )

    print(result.stdout)
    if result.stderr.strip():
        log.warning("ED validation stderr:\n%s", result.stderr)

    if result.returncode != 0:
        log.warning(
            "ED validate.sql exited with code %d — some row counts may
            result.returncode,
        )
    else:
        log.info("ED validation completed — all row counts match expec
```

MIMIC-IV Concepts

The `concepts_postgres/` directory in mimic-code contains SQL scripts that extract clinically meaningful derived tables from the raw MIMIC-IV data. These are built into the **`mimiciv_derived`** schema and cover ~65 concepts across:

Category	Examples
Demographics	`age`, `icustay_detail`, `weight_durations`
Measurements	`vitalsign`, `bg`, `gcs`, `chemistry`, `complete_blood_count`
Medications	`norepinephrine`, `vasopressin`, `antibiotic`, `dobutamine`
Treatments	`ventilation`, `rrt`, `invasive_line`
Organ failure	`kdigo_creatinine`, `kdigo_stages`, `meld`
Severity scores	`sofa`, `sapsii`, `oasis`, `apsiii`, `lods`, `sirs`

Sepsis	`suspicion_of_infection`, `sepsis3`
First-day summaries	`first_day_vitalsign`, `first_day_lab`, `first_day_sofa`

Full description: Pollard et al., JAMIA 2018

Step 16 — Create `mimiciv_derived` Schema + Helper Functions

In [20]:
```
def step16_create_concepts_schema() -> None:
    log.info("=" * 60)
    log.info("STEP 16: Creating mimiciv_derived schema + helper functi
    log.info("=" * 60)

    # Create the schema if it doesn't exist (idempotent)
    with db_connect() as conn:
        cur = conn.cursor()
        cur.execute("CREATE SCHEMA IF NOT EXISTS mimiciv_derived")
        conn.commit()
    log.info("mimiciv_derived schema is ready.")

    # Step 16a: run postgres-functions.sql (BigQuery compatibility hel
    log.info("Step 16a: Installing helper functions (postgres-function

    functions_script = MIMIC_CONCEPTS_DIR / "postgres-functions.sql"
    if not functions_script.exists():
        raise RuntimeError(
            f"Script not found: {functions_script}\n"
            f"Ensure mimic-code is cloned at MIMIC_CODE_DIR='{MIMIC_CO
        )

    # Idempotency check: skip if regexp_extract already exists in mimi
    with db_connect() as conn:
        cur = conn.cursor()
        cur.execute("""
            SELECT COUNT(*)
            FROM pg_proc p
            JOIN pg_namespace n ON n.oid = p.pronamespace
            WHERE n.nspname = 'mimiciv_derived'
              AND p.proname = 'regexp_extract'
        """)
        fn_exists = cur.fetchone()[0]

    if fn_exists:
        log.info("SKIP: Helper function 'regexp_extract' already exist
    else:
        run_psql_in_docker(functions_script)
        log.info("Helper functions installed successfully.")
```

Step 17 — Build All Derived Concepts

In [21]:
```
def step17_build_concepts() -> None:
    log.info("=" * 60)
    log.info("STEP 17: Building MIMIC-IV derived concepts (postgres-ma
    log.info("=" * 60)

    script = MIMIC_CONCEPTS_DIR / "postgres-make-concepts.sql"
    if not script.exists():
        raise RuntimeError(
            f"Script not found: {script}\n"
            f"Ensure mimic-code is cloned at MIMIC_CODE_DIR='{MIMIC_CO
        )

    # Warn if mimiciv_ed appears empty — concepts that reference ED wi
    # gracefully via search_path; concepts that don't touch ED will bu
    try:
        with db_connect() as conn:
            cur = conn.cursor()
            cur.execute("SELECT COUNT(*) FROM mimiciv_ed.edstays")
            ed_rows = cur.fetchone()[0]
        if ed_rows == 0:
            log.warning(
                "mimiciv_ed.edstays has 0 rows — concepts that referen
                "may produce empty results. Run steps 11–15 first to l
            )
    except Exception:
        log.warning(
            "Could not query mimiciv_ed.edstays — ED schema may not ex
            "Concepts that reference ED tables will produce empty resu
        )

    # postgres-make-concepts.sql uses \i with relative paths (e.g. \i
    # psql resolves \i relative to its working directory, so we mount
    # concepts_postgres/ directory at /concepts and set --workdir /con
    abs_concepts_dir = MIMIC_CONCEPTS_DIR.resolve()
    conn_str = (
        f"host={DB_CONTAINER_NAME} "
        f"dbname={POSTGRES_DB} "
        f"user={POSTGRES_USER} "
        f"password={POSTGRES_PASSWORD}"
    )

    log.info("Building all concepts — this may take 15–60 minutes ..."
    result = run(
        [
            "docker", "run", "--rm",
            "--network", DOCKER_NETWORK,
            "--workdir", "/concepts",
            "-v", f"{abs_concepts_dir}:/concepts:ro",
            "postgres:17",
```

```
            "psql", conn_str,
            "-v", "ON_ERROR_STOP=1",
            "-f", "/concepts/postgres-make-concepts.sql",
        ],
        check=False,
        capture=True,
    )

    if result.stdout.strip():
        log.info("Concepts build output:\n%s", result.stdout)
    if result.stderr.strip():
        log.warning("Concepts build stderr:\n%s", result.stderr)

    if result.returncode != 0:
        log.warning(
            "postgres-make-concepts.sql exited with code %d — some con
            result.returncode,
        )
    else:
        log.info("All derived concepts built successfully.")
```

Run Full Pipeline

Executes all 17 steps in order. Each step is idempotent — re-running is safe.

In [22]:
```
step1_check_docker()
step2_ensure_network()
step3_build_image()
step4_start_db()
step5_wait_for_db()
step6_create_schema()
step7_load_data()
step8_add_constraints()
step9_create_indexes()
step10_validate()
step11_create_ed_schema()
step12_load_ed_data()
step13_add_ed_constraints()
step14_create_ed_indexes()
step15_validate_ed()
step16_create_concepts_schema()
step17_build_concepts()

print("\n" + "=" * 60)
print("Pipeline complete. MIMIC-IV, MIMIC-IV-ED, and derived concepts
print(f"Connect via: psql -h localhost -p {POSTGRES_PORT} -U {POSTGRES
print("Web UI: http://localhost:28080  (if Adminer container is runnin
print("=" * 60)
```

```
2026-03-12 19:55:11 [INFO    ] ========================================
====================
```

```
2026-03-12 19:55:11 [INFO    ] STEP 1: Verifying Docker is running
2026-03-12 19:55:11 [INFO    ] ========================================
====================
2026-03-12 19:55:11 [INFO    ] Docker is running. Server version: 27.4.
0
2026-03-12 19:55:11 [INFO    ] ========================================
====================
2026-03-12 19:55:11 [INFO    ] STEP 2: Ensuring Docker network 'mimic-n
et' exists
2026-03-12 19:55:11 [INFO    ] ========================================
====================
2026-03-12 19:55:11 [INFO    ] SKIP: Network 'mimic-net' already exist
s.
2026-03-12 19:55:11 [INFO    ] ========================================
====================
2026-03-12 19:55:11 [INFO    ] STEP 3: Building PostgreSQL image 'mimic
-db'
2026-03-12 19:55:11 [INFO    ] ========================================
====================
2026-03-12 19:55:11 [INFO    ] Building image 'mimic-db' from docker/Do
ckerfile.db ...
#0 building with "desktop-linux" instance using docker driver

#1 [internal] load build definition from Dockerfile.db
#1 transferring dockerfile: 715B done
#1 WARN: SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for
sensitive data (ENV "POSTGRES_PASSWORD") (line 15)
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/postgres:17
#2 DONE 0.0s

#3 [internal] load .dockerignore
#3 transferring context: 2B done
#3 DONE 0.0s

#4 [1/1] FROM docker.io/library/postgres:17
#4 CACHED

#5 exporting to image
#5 exporting layers done
#5 writing image sha256:eee9ed816e260299cfb590c928676738f1e2c894d87e9cf
572e84e92ef2d2b2c done
#5 naming to docker.io/library/mimic-db done
#5 DONE 0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desk
top-linux/34l3skbkf0ij0kibb5bau7rue

 1 warning found (use docker --debug to expand):
 - SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for sensit
ive data (ENV "POSTGRES_PASSWORD") (line 15)
```

```
2026-03-12 19:55:13 [INFO    ] Image 'mimic-db' built successfully.
2026-03-12 19:55:13 [INFO    ] ========================================
====================
2026-03-12 19:55:13 [INFO    ] STEP 4: Starting PostgreSQL container 'm
imic_postgres'
2026-03-12 19:55:13 [INFO    ] ========================================
====================
2026-03-12 19:55:13 [INFO    ] Data directory: /Users/mfanti/Masters_Un
iversityOfTexas_Austin/AI_for_HealthCare/LearningProject2/data/db
2026-03-12 19:55:13 [INFO    ] Creating and starting container 'mimic_p
ostgres' ...
f4b55650d13692379028065302ea97906d5e060c75ccd4bd33e14cad61fd9189
2026-03-12 19:55:13 [INFO    ] Container 'mimic_postgres' created and s
tarted.
2026-03-12 19:55:13 [INFO    ] ========================================
====================
2026-03-12 19:55:13 [INFO    ] STEP 5: Waiting for PostgreSQL to accept
connections
2026-03-12 19:55:13 [INFO    ] ========================================
====================
2026-03-12 19:55:13 [INFO    ] Polling every 5 s (up to 30 attempts)
...
2026-03-12 19:55:13 [INFO    ] Not ready yet [1/30]: connection failed:
connection to server at "127.0.0.1", port 5432 failed: server closed th
e connection unexpectedly
        This probably means the server terminated abnormally
        before or while processing the request.
Multiple connection attempts failed. All failures were:
- host: 'localhost', port: 5432, hostaddr: '::1': connection failed: co
nnection to server at "::1", port 5432 failed: server closed the connec
tion unexpectedly
        This probably means the server terminated abnormally
        before or while processing the request.
- host: 'localhost', port: 5432, hostaddr: '127.0.0.1': connection fail
ed: connection to server at "127.0.0.1", port 5432 failed: server close
d the connection unexpectedly
        This probably means the server terminated abnormally
        before or while processing the request. — retrying in 5 s ...
2026-03-12 19:55:19 [INFO    ] PostgreSQL is ready (attempt 2/30).
2026-03-12 19:55:19 [INFO    ] ========================================
====================
2026-03-12 19:55:19 [INFO    ] STEP 6: Creating MIMIC-IV schemas and ta
bles (create.sql)
2026-03-12 19:55:19 [INFO    ] ========================================
====================
2026-03-12 19:55:19 [INFO    ] Found 0/3 required schemas — running cre
ate.sql ...
2026-03-12 19:55:19 [WARNING ] NOTE: create.sql drops and recreates sch
emas. Existing data will be lost.
psql:/script.sql:9: NOTICE:  schema "mimiciv_hosp" does not exist, skip
ping
psql:/script.sql:11: NOTICE:  schema "mimiciv_icu" does not exist, skip
```

```
ping
psql:/script.sql:13: NOTICE:  schema "mimiciv_derived" does not exist,
skipping
psql:/script.sql:22: NOTICE:  table "admissions" does not exist, skippi
ng
psql:/script.sql:43: NOTICE:  table "d_hcpcs" does not exist, skipping
psql:/script.sql:52: NOTICE:  table "diagnoses_icd" does not exist, ski
pping
psql:/script.sql:62: NOTICE:  table "d_icd_diagnoses" does not exist, s
kipping
psql:/script.sql:70: NOTICE:  table "d_icd_procedures" does not exist,
skipping
psql:/script.sql:78: NOTICE:  table "d_labitems" does not exist, skippi
ng
psql:/script.sql:87: NOTICE:  table "drgcodes" does not exist, skipping
psql:/script.sql:99: NOTICE:  table "emar_detail" does not exist, skipp
ing
psql:/script.sql:137: NOTICE:  table "emar" does not exist, skipping
psql:/script.sql:154: NOTICE:  table "hcpcsevents" does not exist, skip
ping
psql:/script.sql:165: NOTICE:  table "labevents" does not exist, skippi
ng
psql:/script.sql:186: NOTICE:  table "microbiologyevents" does not exis
t, skipping
psql:/script.sql:216: NOTICE:  table "omr" does not exist, skipping
psql:/script.sql:226: NOTICE:  table "patients" does not exist, skippin
g
psql:/script.sql:237: NOTICE:  table "pharmacy" does not exist, skippin
g
psql:/script.sql:269: NOTICE:  table "poe_detail" does not exist, skipp
ing
psql:/script.sql:279: NOTICE:  table "poe" does not exist, skipping
psql:/script.sql:296: NOTICE:  table "prescriptions" does not exist, sk
ipping
psql:/script.sql:322: NOTICE:  table "procedures_icd" does not exist, s
kipping
psql:/script.sql:333: NOTICE:  table "provider" does not exist, skippin
g
psql:/script.sql:339: NOTICE:  table "services" does not exist, skippin
g
psql:/script.sql:349: NOTICE:  table "transfers" does not exist, skippi
ng
psql:/script.sql:363: NOTICE:  table "caregiver" does not exist, skippi
ng
psql:/script.sql:369: NOTICE:  table "chartevents" does not exist, skip
ping
psql:/script.sql:385: NOTICE:  table "datetimeevents" does not exist, s
kipping
psql:/script.sql:400: NOTICE:  table "d_items" does not exist, skipping
psql:/script.sql:414: NOTICE:  table "icustays" does not exist, skippin
g
psql:/script.sql:427: NOTICE:  table "ingredientevents" does not exist,
```

```
skipping
psql:/script.sql:449: NOTICE:  table "inputevents" does not exist, skip
ping
psql:/script.sql:480: NOTICE:  table "outputevents" does not exist, ski
pping
psql:/script.sql:494: NOTICE:  table "procedureevents" does not exist,
skipping
2026-03-12 19:55:19 [INFO    ] Schemas and tables created successfully.
2026-03-12 19:55:19 [INFO    ] ========================================
====================
2026-03-12 19:55:19 [INFO    ] STEP 7: Loading MIMIC-IV data (load_gz.s
ql)
2026-03-12 19:55:19 [INFO    ] ========================================
====================
2026-03-12 19:55:19 [INFO    ] NOTE: This step can take many hours for
the full dataset.
2026-03-12 19:55:19 [INFO    ] No data found in mimiciv_hosp.patients -
starting data load ...
DROP SCHEMA
CREATE SCHEMA
DROP SCHEMA
CREATE SCHEMA
DROP SCHEMA
CREATE SCHEMA
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
```

```
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
```

```
2026-03-12 19:55:19 [INFO    ] Truncated 31 tables for clean reload.
2026-03-12 19:55:19 [INFO    ] Data directory: /Volumes/ExternalSSD/AI_
For_Healthcare_files/data
2026-03-12 19:55:19 [INFO    ] This may take many hours. Per-table timi
ng will appear below when complete.
2026-03-12 19:55:19 [INFO    ] Row limits active (from './limits.jso
n'): default=None
2026-03-12 19:55:19 [INFO    ]   admissions                    → sourc
ing from core/ (not in hosp/)
2026-03-12 19:55:19 [INFO    ]   patients                      → sourc
ing from core/ (not in hosp/)
2026-03-12 19:55:19 [INFO    ]   transfers                     → sourc
ing from core/ (not in hosp/)
2026-03-12 19:55:19 [INFO    ] Sourcing 3 table(s) from core/ instead o
f hosp/: admissions, patients, transfers
2026-03-12 20:30:19 [INFO    ]   admissions                     starte
d  2026-03-12 23:55:19
2026-03-12 20:30:19 [INFO    ]   admissions                     finish
```

```
ed 2026-03-12 23:55:21  (2.0 s)
2026-03-12 20:30:19 [INFO    ]   d_hcpcs                          starte
d  2026-03-12 23:55:21
2026-03-12 20:30:19 [INFO    ]   d_hcpcs                          finish
ed 2026-03-12 23:55:21  (0.0 s)
2026-03-12 20:30:19 [INFO    ]   diagnoses_icd                    starte
d  2026-03-12 23:55:21
2026-03-12 20:30:19 [INFO    ]   diagnoses_icd                    finish
ed 2026-03-12 23:55:26  (5.0 s)
2026-03-12 20:30:19 [INFO    ]   d_icd_diagnoses                  starte
d  2026-03-12 23:55:26
2026-03-12 20:30:19 [INFO    ]   d_icd_diagnoses                  finish
ed 2026-03-12 23:55:26  (0.0 s)
2026-03-12 20:30:19 [INFO    ]   d_icd_procedures                 starte
d  2026-03-12 23:55:26
2026-03-12 20:30:19 [INFO    ]   d_icd_procedures                 finish
ed 2026-03-12 23:55:26  (0.0 s)
2026-03-12 20:30:19 [INFO    ]   d_labitems                       starte
d  2026-03-12 23:55:26
2026-03-12 20:30:19 [INFO    ]   d_labitems                       finish
ed 2026-03-12 23:55:26  (0.0 s)
2026-03-12 20:30:19 [INFO    ]   drgcodes                         starte
d  2026-03-12 23:55:26
2026-03-12 20:30:19 [INFO    ]   drgcodes                         finish
ed 2026-03-12 23:55:27  (1.0 s)
2026-03-12 20:30:19 [INFO    ]   emar_detail                      starte
d  2026-03-12 23:55:27
2026-03-12 20:30:19 [INFO    ]   emar_detail                      finish
ed 2026-03-12 23:58:16  (169.0 s)
2026-03-12 20:30:19 [INFO    ]   emar                             starte
d  2026-03-12 23:58:16
2026-03-12 20:30:19 [INFO    ]   emar                             finish
ed 2026-03-13 00:00:12  (116.0 s)
2026-03-12 20:30:19 [INFO    ]   hcpcsevents                      starte
d  2026-03-13 00:00:12
2026-03-12 20:30:19 [INFO    ]   hcpcsevents                      finish
ed 2026-03-13 00:00:12  (0.0 s)
2026-03-12 20:30:19 [INFO    ]   labevents                        starte
d  2026-03-13 00:00:12
2026-03-12 20:30:19 [INFO    ]   labevents                        finish
ed 2026-03-13 00:07:12  (420.0 s)
2026-03-12 20:30:19 [INFO    ]   microbiologyevents               starte
d  2026-03-13 00:07:12
2026-03-12 20:30:19 [INFO    ]   microbiologyevents               finish
ed 2026-03-13 00:07:30  (18.0 s)
2026-03-12 20:30:19 [INFO    ]   omr                              starte
d  2026-03-13 00:07:30
2026-03-12 20:30:19 [INFO    ]   omr                              finish
ed 2026-03-13 00:07:38  (8.0 s)
2026-03-12 20:30:19 [INFO    ]   patients                         starte
d  2026-03-13 00:07:38
2026-03-12 20:30:19 [INFO    ]   patients                         finish
```

```
ed 2026-03-13 00:07:38  (0.0 s)
2026-03-12 20:30:19 [INFO    ]   pharmacy                        starte
d  2026-03-13 00:07:38
2026-03-12 20:30:19 [INFO    ]   pharmacy                        finish
ed 2026-03-13 00:09:00  (82.0 s)
2026-03-12 20:30:19 [INFO    ]   poe_detail                      starte
d  2026-03-13 00:09:00
2026-03-12 20:30:19 [INFO    ]   poe_detail                      finish
ed 2026-03-13 00:09:11  (11.0 s)
2026-03-12 20:30:19 [INFO    ]   poe                             starte
d  2026-03-13 00:09:11
2026-03-12 20:30:19 [INFO    ]   poe                             finish
ed 2026-03-13 00:11:08  (117.0 s)
2026-03-12 20:30:19 [INFO    ]   prescriptions                   starte
d  2026-03-13 00:11:08
2026-03-12 20:30:19 [INFO    ]   prescriptions                   finish
ed 2026-03-13 00:12:27  (79.0 s)
2026-03-12 20:30:19 [INFO    ]   procedures_icd                  starte
d  2026-03-13 00:12:27
2026-03-12 20:30:19 [INFO    ]   procedures_icd                  finish
ed 2026-03-13 00:12:28  (1.0 s)
2026-03-12 20:30:19 [INFO    ]   provider                        starte
d  2026-03-13 00:12:28
2026-03-12 20:30:19 [INFO    ]   provider                        finish
ed 2026-03-13 00:12:28  (0.0 s)
2026-03-12 20:30:19 [INFO    ]   services                        starte
d  2026-03-13 00:12:28
2026-03-12 20:30:19 [INFO    ]   services                        finish
ed 2026-03-13 00:12:28  (0.0 s)
2026-03-12 20:30:19 [INFO    ]   transfers                       starte
d  2026-03-13 00:12:28
2026-03-12 20:30:19 [INFO    ]   transfers                       finish
ed 2026-03-13 00:12:32  (4.0 s)
2026-03-12 20:30:19 [INFO    ]   caregiver                       starte
d  2026-03-13 00:12:32
2026-03-12 20:30:19 [INFO    ]   caregiver                       finish
ed 2026-03-13 00:12:32  (0.0 s)
2026-03-12 20:30:19 [INFO    ]   chartevents                     starte
d  2026-03-13 00:12:32
2026-03-12 20:30:19 [INFO    ]   chartevents                     finish
ed 2026-03-13 00:27:44  (912.0 s)
2026-03-12 20:30:19 [INFO    ]   datetimeevents                  starte
d  2026-03-13 00:27:44
2026-03-12 20:30:19 [INFO    ]   datetimeevents                  finish
ed 2026-03-13 00:28:06  (22.0 s)
2026-03-12 20:30:19 [INFO    ]   d_items                         starte
d  2026-03-13 00:28:06
2026-03-12 20:30:19 [INFO    ]   d_items                         finish
ed 2026-03-13 00:28:06  (0.0 s)
2026-03-12 20:30:19 [INFO    ]   icustays                        starte
d  2026-03-13 00:28:06
2026-03-12 20:30:19 [INFO    ]   icustays                        finish
```

```
ed 2026-03-13 00:28:06  (0.0 s)
2026-03-12 20:30:19 [INFO    ]   ingredientevents               starte
d  2026-03-13 00:28:06
2026-03-12 20:30:19 [INFO    ]   ingredientevents               finish
ed 2026-03-13 00:29:01  (55.0 s)
2026-03-12 20:30:19 [INFO    ]   inputevents                    starte
d  2026-03-13 00:29:01
2026-03-12 20:30:19 [INFO    ]   inputevents                    finish
ed 2026-03-13 00:30:05  (64.0 s)
2026-03-12 20:30:19 [INFO    ]   outputevents                   starte
d  2026-03-13 00:30:05
2026-03-12 20:30:19 [INFO    ]   outputevents                   finish
ed 2026-03-13 00:30:16  (11.0 s)
2026-03-12 20:30:19 [INFO    ]   procedureevents                starte
d  2026-03-13 00:30:16
2026-03-12 20:30:19 [INFO    ]   procedureevents                finish
ed 2026-03-13 00:30:19  (3.0 s)
2026-03-12 20:30:19 [INFO    ] Data loaded successfully.
2026-03-12 20:30:19 [INFO    ] ========================================
====================
2026-03-12 20:30:19 [INFO    ] STEP 8: Adding primary key constraints (
constraint.sql)
2026-03-12 20:30:19 [INFO    ] ========================================
====================
2026-03-12 20:30:20 [INFO    ] Row limits active — adding FK constraint
s with NOT VALID (skips data scan).
psql:/script.sql:13: NOTICE:  constraint "admissions_pk" of relation "a
dmissions" does not exist, skipping
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
psql:/script.sql:20: NOTICE:  constraint "d_hcpcs_pk" of relation "d_hc
pcs" does not exist, skipping
psql:/script.sql:27: NOTICE:  constraint "diagnoses_icd_pk" of relation
"diagnoses_icd" does not exist, skipping
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
psql:/script.sql:34: NOTICE:  constraint "d_icd_diagnoses_pk" of relati
on "d_icd_diagnoses" does not exist, skipping
psql:/script.sql:41: NOTICE:  constraint "d_icd_procedures_pk" of relat
ion "d_icd_procedures" does not exist, skipping
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
```

```
psql:/script.sql:48: NOTICE:  constraint "d_labitems_pk" of relation "d
_labitems" does not exist, skipping
psql:/script.sql:62: NOTICE:  constraint "emar_pk" of relation "emar" d
oes not exist, skipping
ALTER TABLE
ALTER TABLE
ALTER TABLE
psql:/script.sql:69: NOTICE:  constraint "hcpcsevents_pk" of relation "
hcpcsevents" does not exist, skipping
psql:/script.sql:76: NOTICE:  constraint "labevents_pk" of relation "la
bevents" does not exist, skipping
ALTER TABLE
ALTER TABLE
ALTER TABLE
psql:/script.sql:83: NOTICE:  constraint "microbiologyevents_pk" of rel
ation "microbiologyevents" does not exist, skipping
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
psql:/script.sql:90: NOTICE:  constraint "patients_pk" of relation "pat
ients" does not exist, skipping
psql:/script.sql:97: NOTICE:  constraint "pharmacy_pk" of relation "pha
rmacy" does not exist, skipping
ALTER TABLE
ALTER TABLE
psql:/script.sql:104: NOTICE:  constraint "poe_detail_pk" of relation "
poe_detail" does not exist, skipping
ALTER TABLE
ALTER TABLE
psql:/script.sql:111: NOTICE:  constraint "poe_pk" of relation "poe" do
es not exist, skipping
ALTER TABLE
ALTER TABLE
psql:/script.sql:118: NOTICE:  constraint "prescriptions_pk" of relatio
n "prescriptions" does not exist, skipping
ALTER TABLE
ALTER TABLE
psql:/script.sql:125: NOTICE:  constraint "procedures_icd_pk" of relati
on "procedures_icd" does not exist, skipping
ALTER TABLE
ALTER TABLE
psql:/script.sql:132: NOTICE:  constraint "services_pk" of relation "se
rvices" does not exist, skipping
ALTER TABLE
ALTER TABLE
```

```
psql:/script.sql:143: NOTICE:  constraint "datetimeevents_pk" of relati
on "datetimeevents" does not exist, skipping
psql:/script.sql:150: NOTICE:  constraint "d_items_pk" of relation "d_i
tems" does not exist, skipping
psql:/script.sql:157: NOTICE:  constraint "icustays_pk" of relation "ic
ustays" does not exist, skipping
psql:/script.sql:164: NOTICE:  constraint "inputevents_pk" of relation
"inputevents" does not exist, skipping
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
psql:/script.sql:171: NOTICE:  constraint "outputevents_pk" of relation
"outputevents" does not exist, skipping
ALTER TABLE
ALTER TABLE
psql:/script.sql:178: NOTICE:  constraint "procedureevents_pk" of relat
ion "procedureevents" does not exist, skipping
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
psql:/script.sql:195: NOTICE:  constraint "admissions_patients_fk" of r
elation "admissions" does not exist, skipping
psql:/script.sql:203: NOTICE:  constraint "diagnoses_icd_patients_fk" o
f relation "diagnoses_icd" does not exist, skipping
```

```
psql:/script.sql:209: NOTICE:  constraint "diagnoses_icd_admissions_fk"
of relation "diagnoses_icd" does not exist, skipping
psql:/script.sql:217: NOTICE:  constraint "drgcodes_patients_fk" of rel
ation "drgcodes" does not exist, skipping
psql:/script.sql:223: NOTICE:  constraint "drgcodes_admissions_fk" of r
elation "drgcodes" does not exist, skipping
psql:/script.sql:231: NOTICE:  constraint "emar_detail_patients_fk" of
relation "emar_detail" does not exist, skipping
psql:/script.sql:237: NOTICE:  constraint "emar_detail_emar_fk" of rela
tion "emar_detail" does not exist, skipping
psql:/script.sql:245: NOTICE:  constraint "emar_patients_fk" of relatio
n "emar" does not exist, skipping
psql:/script.sql:251: NOTICE:  constraint "emar_admissions_fk" of relat
ion "emar" does not exist, skipping
psql:/script.sql:259: NOTICE:  constraint "hcpcsevents_patients_fk" of
relation "hcpcsevents" does not exist, skipping
psql:/script.sql:265: NOTICE:  constraint "hcpcsevents_admissions_fk" o
f relation "hcpcsevents" does not exist, skipping
psql:/script.sql:271: NOTICE:  constraint "hcpcsevents_d_hcpcs_fk" of r
elation "hcpcsevents" does not exist, skipping
psql:/script.sql:279: NOTICE:  constraint "labevents_patients_fk" of re
lation "labevents" does not exist, skipping
psql:/script.sql:285: NOTICE:  constraint "labevents_d_labitems_fk" of
relation "labevents" does not exist, skipping
psql:/script.sql:293: NOTICE:  constraint "microbiologyevents_patients_
fk" of relation "microbiologyevents" does not exist, skipping
psql:/script.sql:299: NOTICE:  constraint "microbiologyevents_admission
s_fk" of relation "microbiologyevents" does not exist, skipping
psql:/script.sql:307: NOTICE:  constraint "pharmacy_patients_fk" of rel
ation "pharmacy" does not exist, skipping
psql:/script.sql:313: NOTICE:  constraint "pharmacy_admissions_fk" of r
elation "pharmacy" does not exist, skipping
psql:/script.sql:321: NOTICE:  constraint "poe_detail_patients_fk" of r
elation "poe_detail" does not exist, skipping
psql:/script.sql:327: NOTICE:  constraint "poe_detail_poe_fk" of relati
on "poe_detail" does not exist, skipping
psql:/script.sql:335: NOTICE:  constraint "poe_patients_fk" of relation
"poe" does not exist, skipping
psql:/script.sql:341: NOTICE:  constraint "poe_admissions_fk" of relati
on "poe" does not exist, skipping
psql:/script.sql:349: NOTICE:  constraint "prescriptions_patients_fk" o
f relation "prescriptions" does not exist, skipping
psql:/script.sql:355: NOTICE:  constraint "prescriptions_admissions_fk"
of relation "prescriptions" does not exist, skipping
psql:/script.sql:363: NOTICE:  constraint "procedures_icd_patients_fk"
of relation "procedures_icd" does not exist, skipping
psql:/script.sql:369: NOTICE:  constraint "procedures_icd_admissions_f
k" of relation "procedures_icd" does not exist, skipping
psql:/script.sql:377: NOTICE:  constraint "services_patients_fk" of rel
ation "services" does not exist, skipping
psql:/script.sql:383: NOTICE:  constraint "services_admissions_fk" of r
elation "services" does not exist, skipping
```

```
psql:/script.sql:391: NOTICE:  constraint "transfers_pk" of relation "t
ransfers" does not exist, skipping
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
```

```
psql:/script.sql:398: NOTICE:  constraint "transfers_patients_fk" of re
lation "transfers" does not exist, skipping
psql:/script.sql:411: NOTICE:  constraint "chartevents_patients_fk" of
relation "chartevents" does not exist, skipping
psql:/script.sql:417: NOTICE:  constraint "chartevents_admissions_fk" o
f relation "chartevents" does not exist, skipping
psql:/script.sql:423: NOTICE:  constraint "chartevents_icustays_fk" of
relation "chartevents" does not exist, skipping
psql:/script.sql:429: NOTICE:  constraint "chartevents_d_items_fk" of r
elation "chartevents" does not exist, skipping
psql:/script.sql:437: NOTICE:  constraint "datetimeevents_patients_fk"
of relation "datetimeevents" does not exist, skipping
psql:/script.sql:443: NOTICE:  constraint "datetimeevents_admissions_f
k" of relation "datetimeevents" does not exist, skipping
psql:/script.sql:449: NOTICE:  constraint "datetimeevents_icustays_fk"
of relation "datetimeevents" does not exist, skipping
psql:/script.sql:455: NOTICE:  constraint "datetimeevents_d_items_fk" o
f relation "datetimeevents" does not exist, skipping
psql:/script.sql:463: NOTICE:  constraint "icustays_patients_fk" of rel
ation "icustays" does not exist, skipping
psql:/script.sql:469: NOTICE:  constraint "icustays_admissions_fk" of r
elation "icustays" does not exist, skipping
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
psql:/script.sql:477: NOTICE:  constraint "inputevents_patients_fk" of
relation "inputevents" does not exist, skipping
psql:/script.sql:483: NOTICE:  constraint "inputevents_admissions_fk" o
f relation "inputevents" does not exist, skipping
psql:/script.sql:489: NOTICE:  constraint "inputevents_icustays_fk" of
relation "inputevents" does not exist, skipping
psql:/script.sql:495: NOTICE:  constraint "inputevents_d_items_fk" of r
elation "inputevents" does not exist, skipping
```

```
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
psql:/script.sql:503: NOTICE:  constraint "outputevents_patients_fk" of
relation "outputevents" does not exist, skipping
psql:/script.sql:509: NOTICE:  constraint "outputevents_admissions_fk"
of relation "outputevents" does not exist, skipping
psql:/script.sql:515: NOTICE:  constraint "outputevents_icustays_fk" of
relation "outputevents" does not exist, skipping
psql:/script.sql:521: NOTICE:  constraint "outputevents_d_items_fk" of
relation "outputevents" does not exist, skipping
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
psql:/script.sql:529: NOTICE:  constraint "procedureevents_patients_fk"
of relation "procedureevents" does not exist, skipping
psql:/script.sql:535: NOTICE:  constraint "procedureevents_admissions_f
k" of relation "procedureevents" does not exist, skipping
psql:/script.sql:541: NOTICE:  constraint "procedureevents_icustays_fk"
of relation "procedureevents" does not exist, skipping
psql:/script.sql:547: NOTICE:  constraint "procedureevents_d_items_fk"
of relation "procedureevents" does not exist, skipping
2026-03-12 20:38:28 [INFO    ] Constraints added successfully.
2026-03-12 20:38:28 [INFO    ] ========================================
====================
2026-03-12 20:38:28 [INFO    ] STEP 9: Creating indexes (index.sql)
2026-03-12 20:38:28 [INFO    ] ========================================
====================
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
2026-03-12 20:38:28 [INFO    ] Indexes not found — running index.sql
...
SET
DROP INDEX
```

```
psql:/script.sql:15: NOTICE:  index "admissions_idx01" does not exist,
skipping
CREATE INDEX
DROP INDEX
psql:/script.sql:21: NOTICE:  index "d_icd_diag_idx02" does not exist,
skipping
psql:/script.sql:27: NOTICE:  index "d_icd_proc_idx02" does not exist,
skipping
CREATE INDEX
DROP INDEX
CREATE INDEX
DROP INDEX
psql:/script.sql:33: NOTICE:  index "drgcodes_idx01" does not exist, sk
ipping
CREATE INDEX
DROP INDEX
psql:/script.sql:37: NOTICE:  index "drgcodes_idx02" does not exist, sk
ipping
CREATE INDEX
DROP INDEX
CREATE INDEX
DROP INDEX
psql:/script.sql:43: NOTICE:  index "d_labitems_idx01" does not exist,
skipping
psql:/script.sql:49: NOTICE:  index "emar_detail_idx01" does not exist,
skipping
psql:/script.sql:53: NOTICE:  index "emar_detail_idx02" does not exist,
skipping
CREATE INDEX
DROP INDEX
CREATE INDEX
DROP INDEX
psql:/script.sql:57: NOTICE:  index "emar_detail_idx03" does not exist,
skipping
CREATE INDEX
DROP INDEX
psql:/script.sql:61: NOTICE:  index "emar_det_idx04" does not exist, sk
ipping
CREATE INDEX
DROP INDEX
psql:/script.sql:67: NOTICE:  index "emar_idx01" does not exist, skippi
ng
CREATE INDEX
DROP INDEX
psql:/script.sql:71: NOTICE:  index "emar_idx02" does not exist, skippi
ng
CREATE INDEX
DROP INDEX
psql:/script.sql:75: NOTICE:  index "emar_idx03" does not exist, skippi
ng
```

```
CREATE INDEX
DROP INDEX
psql:/script.sql:79: NOTICE:  index "emar_idx04" does not exist, skippi
ng
CREATE INDEX
DROP INDEX
CREATE INDEX
DROP INDEX
psql:/script.sql:85: NOTICE:  index "hcpcsevents_idx04" does not exist,
skipping
psql:/script.sql:91: NOTICE:  index "labevents_idx01" does not exist, s
kipping
CREATE INDEX
DROP INDEX
psql:/script.sql:95: NOTICE:  index "labevents_idx02" does not exist, s
kipping
CREATE INDEX
DROP INDEX
psql:/script.sql:101: NOTICE:  index "microbiologyevents_idx01" does no
t exist, skipping
psql:/script.sql:105: NOTICE:  index "microbiologyevents_idx02" does no
t exist, skipping
CREATE INDEX
DROP INDEX
CREATE INDEX
DROP INDEX
psql:/script.sql:109: NOTICE:  index "microbiologyevents_idx03" does no
t exist, skipping
CREATE INDEX
DROP INDEX
CREATE INDEX
DROP INDEX
psql:/script.sql:114: NOTICE:  index "patients_idx01" does not exist, s
kipping
psql:/script.sql:118: NOTICE:  index "patients_idx02" does not exist, s
kipping
CREATE INDEX
DROP INDEX
psql:/script.sql:124: NOTICE:  index "pharmacy_idx01" does not exist, s
kipping
CREATE INDEX
DROP INDEX
psql:/script.sql:128: NOTICE:  index "pharmacy_idx02" does not exist, s
kipping
psql:/script.sql:132: NOTICE:  index "pharmacy_idx03" does not exist, s
kipping
CREATE INDEX
DROP INDEX
CREATE INDEX
DROP INDEX
```

```
psql:/script.sql:136: NOTICE:  index "pharmacy_idx04" does not exist, s
kipping
psql:/script.sql:142: NOTICE:  index "poe_idx01" does not exist, skippi
ng
CREATE INDEX
DROP INDEX
psql:/script.sql:148: NOTICE:  index "prescriptions_idx01" does not exi
st, skipping
CREATE INDEX
DROP INDEX
CREATE INDEX
DROP INDEX
psql:/script.sql:154: NOTICE:  index "transfers_idx01" does not exist,
skipping
CREATE INDEX
DROP INDEX
psql:/script.sql:158: NOTICE:  index "transfers_idx02" does not exist,
skipping
CREATE INDEX
DROP INDEX
psql:/script.sql:162: NOTICE:  index "transfers_idx03" does not exist,
skipping
CREATE INDEX
SET
DROP INDEX
psql:/script.sql:174: NOTICE:  index "chartevents_idx01" does not exis
t, skipping
CREATE INDEX
DROP INDEX
psql:/script.sql:180: NOTICE:  index "datetimeevents_idx01" does not ex
ist, skipping
CREATE INDEX
DROP INDEX
psql:/script.sql:184: NOTICE:  index "datetimeevents_idx02" does not ex
ist, skipping
CREATE INDEX
DROP INDEX
CREATE INDEX
DROP INDEX
CREATE INDEX
DROP INDEX
CREATE INDEX
DROP INDEX
```

```
psql:/script.sql:190: NOTICE:  index "d_items_idx01" does not exist, sk
ipping
psql:/script.sql:194: NOTICE:  index "d_items_idx02" does not exist, sk
ipping
psql:/script.sql:200: NOTICE:  index "icustays_idx01" does not exist, s
kipping
psql:/script.sql:204: NOTICE:  index "icustays_idx02" does not exist, s
kipping
CREATE INDEX
DROP INDEX
psql:/script.sql:210: NOTICE:  index "inputevents_idx01" does not exis
t, skipping
CREATE INDEX
DROP INDEX
psql:/script.sql:214: NOTICE:  index "inputevents_idx02" does not exis
t, skipping
CREATE INDEX
DROP INDEX
psql:/script.sql:220: NOTICE:  index "outputevents_idx01" does not exis
t, skipping
CREATE INDEX
DROP INDEX
psql:/script.sql:226: NOTICE:  index "procedureevents_idx01" does not e
xist, skipping
CREATE INDEX
DROP INDEX
psql:/script.sql:230: NOTICE:  index "procedureevents_idx02" does not e
xist, skipping
2026-03-12 21:00:08 [INFO    ] Indexes created successfully.
2026-03-12 21:00:08 [INFO    ] ========================================
====================
2026-03-12 21:00:08 [INFO    ] STEP 10: Validating row counts (validat
e.sql)
2026-03-12 21:00:08 [INFO    ] ========================================
====================
CREATE INDEX
2026-03-12 21:01:03 [INFO    ] Validation completed — all row counts ma
tch expected values.
2026-03-12 21:01:03 [INFO    ] ========================================
====================
2026-03-12 21:01:03 [INFO    ] STEP 11: Creating MIMIC-IV-ED schema and
tables (ED create.sql)
2026-03-12 21:01:03 [INFO    ] ========================================
====================
2026-03-12 21:01:03 [INFO    ] mimiciv_ed schema not found — running ED
create.sql ...
2026-03-12 21:01:03 [WARNING ] NOTE: ED create.sql drops and recreates
mimiciv_ed. Existing ED data will be lost.
```

```
        tbl         | expected_count | observed_count | row_count_check
--------------------+----------------+----------------+----------------
-
 admissions         |         546028 |         546028 | PASSED
 chartevents        |      432997491 |      432997491 | PASSED
 datetimeevents     |        9979761 |        9979761 | PASSED
 d_hcpcs            |          89208 |          89208 | PASSED
 diagnoses_icd      |        6364488 |        6364488 | PASSED
 d_icd_diagnoses    |         112107 |         112107 | PASSED
 d_icd_procedures   |          86423 |          86423 | PASSED
 d_items            |           4095 |           4095 | PASSED
 d_labitems         |           1650 |           1650 | PASSED
 drgcodes           |         761856 |         761856 | PASSED
 emar               |       42808593 |       42808593 | PASSED
 emar_detail        |       87371064 |       87371064 | PASSED
 hcpcsevents        |         186074 |         186074 | PASSED
 icustays           |          94458 |          94458 | PASSED
 inputevents        |       10953713 |       10953713 | PASSED
 labevents          |      158374764 |      158374764 | PASSED
 microbiologyevents |        3988224 |        3988224 | PASSED
 omr                |        7753027 |        7753027 | PASSED
 outputevents       |        5359395 |        5359395 | PASSED
 patients           |         364627 |         364627 | PASSED
 pharmacy           |       17847567 |       17847567 | PASSED
 poe                |       52212109 |       52212109 | PASSED
 poe_detail         |        8504982 |        8504982 | PASSED
 prescriptions      |       20292611 |       20292611 | PASSED
 procedureevents    |         808706 |         808706 | PASSED
 procedures_icd     |         859655 |         859655 | PASSED
 services           |         593071 |         593071 | PASSED
 transfers          |        2413581 |        2413581 | PASSED
(28 rows)
```

```
psql:/script.sql:11: NOTICE:  schema "mimiciv_ed" does not exist, skipp
ing
psql:/script.sql:26: NOTICE:  table "diagnosis" does not exist, skippin
g
psql:/script.sql:41: NOTICE:  table "edstays" does not exist, skipping
psql:/script.sql:59: NOTICE:  table "medrecon" does not exist, skipping
psql:/script.sql:77: NOTICE:  table "pyxis" does not exist, skipping
psql:/script.sql:93: NOTICE:  table "triage" does not exist, skipping
psql:/script.sql:113: NOTICE:  table "vitalsign" does not exist, skippi
ng
2026-03-12 21:01:04 [INFO    ] ED schema and tables created successfull
y.
2026-03-12 21:01:04 [INFO    ] ========================================
====================
2026-03-12 21:01:04 [INFO    ] STEP 12: Loading MIMIC-IV-ED data (ED lo
ad_gz.sql)
2026-03-12 21:01:04 [INFO    ] ========================================
====================
2026-03-12 21:01:04 [INFO    ] No data found in mimiciv_ed.edstays — st
arting ED data load ...
2026-03-12 21:01:04 [INFO    ] Truncated 6 tables for clean reload.
2026-03-12 21:01:04 [INFO    ] ED data directory: /Volumes/ExternalSSD/
AI_For_Healthcare_files/data/ed
2026-03-12 21:01:04 [INFO    ] Row limits active (from './limits.jso
n'): default=None
DROP SCHEMA
CREATE SCHEMA
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
```

```
2026-03-12 21:01:18 [INFO    ]   diagnosis                       starte
d  2026-03-13 01:01:04
2026-03-12 21:01:18 [INFO    ]   diagnosis                       finish
ed 2026-03-13 01:01:05  (1.0 s)
2026-03-12 21:01:18 [INFO    ]   edstays                         starte
d  2026-03-13 01:01:05
2026-03-12 21:01:18 [INFO    ]   edstays                         finish
ed 2026-03-13 01:01:06  (1.0 s)
2026-03-12 21:01:18 [INFO    ]   medrecon                        starte
d  2026-03-13 01:01:06
2026-03-12 21:01:18 [INFO    ]   medrecon                        finish
ed 2026-03-13 01:01:12  (6.0 s)
2026-03-12 21:01:18 [INFO    ]   pyxis                           starte
d  2026-03-13 01:01:12
2026-03-12 21:01:18 [INFO    ]   pyxis                           finish
ed 2026-03-13 01:01:14  (2.0 s)
2026-03-12 21:01:18 [INFO    ]   triage                          starte
d  2026-03-13 01:01:14
2026-03-12 21:01:18 [INFO    ]   triage                          finish
ed 2026-03-13 01:01:15  (1.0 s)
2026-03-12 21:01:18 [INFO    ]   vitalsign                       starte
d  2026-03-13 01:01:15
2026-03-12 21:01:18 [INFO    ]   vitalsign                       finish
ed 2026-03-13 01:01:18  (3.0 s)
2026-03-12 21:01:18 [INFO    ] ED data loaded successfully.
2026-03-12 21:01:18 [INFO    ] ========================================
====================
2026-03-12 21:01:18 [INFO    ] STEP 13: Adding MIMIC-IV-ED constraints
(ED constraint.sql)
2026-03-12 21:01:18 [INFO    ] ========================================
====================
2026-03-12 21:01:18 [INFO    ] Row limits active — adding ED FK constra
ints with NOT VALID (skips data scan).
SET
ALTER TABLE
psql:/script.sql:8: NOTICE:  constraint "edstays_pk" of relation "edsta
ys" does not exist, skipping
ALTER TABLE
ALTER TABLE
psql:/script.sql:13: NOTICE:  constraint "diagnosis_pk" of relation "di
agnosis" does not exist, skipping
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
psql:/script.sql:28: NOTICE:  constraint "triage_pk" of relation "triag
e" does not exist, skipping
psql:/script.sql:33: NOTICE:  constraint "vitalsign_pk" of relation "vi
talsign" does not exist, skipping
ALTER TABLE
```

```
psql:/script.sql:44: NOTICE:  constraint "diagnosis_edstays_fk" of rela
tion "diagnosis" does not exist, skipping
ALTER TABLE
ALTER TABLE
psql:/script.sql:50: NOTICE:  constraint "medrecon_edstays_fk" of relat
ion "medrecon" does not exist, skipping
ALTER TABLE
ALTER TABLE
psql:/script.sql:56: NOTICE:  constraint "pyxis_edstays_fk" of relation
"pyxis" does not exist, skipping
ALTER TABLE
ALTER TABLE
psql:/script.sql:62: NOTICE:  constraint "triage_edstays_fk" of relatio
n "triage" does not exist, skipping
ALTER TABLE
ALTER TABLE
psql:/script.sql:68: NOTICE:  constraint "vitalsign_edstays_fk" of rela
tion "vitalsign" does not exist, skipping
2026-03-12 21:01:26 [INFO    ] ED constraints added successfully.
2026-03-12 21:01:26 [INFO    ] ========================================
====================
2026-03-12 21:01:26 [INFO    ] STEP 14: Creating MIMIC-IV-ED indexes (E
D index.sql)
2026-03-12 21:01:26 [INFO    ] ========================================
====================
2026-03-12 21:01:26 [INFO    ] ED indexes not found — running ED index.
sql ...
ALTER TABLE
ALTER TABLE
SET
DROP INDEX
psql:/script.sql:11: NOTICE:  index "diagnosis_idx01" does not exist, s
kipping
CREATE INDEX
DROP INDEX
psql:/script.sql:15: NOTICE:  index "diagnosis_idx02" does not exist, s
kipping
CREATE INDEX
DROP INDEX
psql:/script.sql:21: NOTICE:  index "edstays_idx01" does not exist, ski
pping
CREATE INDEX
DROP INDEX
psql:/script.sql:25: NOTICE:  index "edstays_idx02" does not exist, ski
pping
CREATE INDEX
DROP INDEX
psql:/script.sql:31: NOTICE:  index "medrecon_idx01" does not exist, sk
ipping
```

```
CREATE INDEX
DROP INDEX
psql:/script.sql:37: NOTICE:  index "pyxis_idx01" does not exist, skipp
ing
CREATE INDEX
DROP INDEX
psql:/script.sql:41: NOTICE:  index "pyxis_idx02" does not exist, skipp
ing
CREATE INDEX
DROP INDEX
CREATE INDEX
DROP INDEX
psql:/script.sql:47: NOTICE:  index "triage_idx01" does not exist, skip
ping
psql:/script.sql:53: NOTICE:  index "vitalsign_idx01" does not exist, s
kipping
2026-03-12 21:01:36 [INFO    ] ED indexes created successfully.
2026-03-12 21:01:36 [INFO    ] ========================================
====================
2026-03-12 21:01:36 [INFO    ] STEP 15: Validating MIMIC-IV-ED row coun
ts (ED validate.sql)
2026-03-12 21:01:36 [INFO    ] ========================================
====================
CREATE INDEX
2026-03-12 21:01:37 [INFO    ] ED validation completed — all row counts
match expected values.
2026-03-12 21:01:37 [INFO    ] ========================================
====================
2026-03-12 21:01:37 [INFO    ] STEP 16: Creating mimiciv_derived schema
+ helper functions
2026-03-12 21:01:37 [INFO    ] ========================================
====================
2026-03-12 21:01:37 [INFO    ] mimiciv_derived schema is ready.
2026-03-12 21:01:37 [INFO    ] Step 16a: Installing helper functions (p
ostgres-functions.sql) ...
    tbl    | expected_count | observed_count | row_count_check
-----------+----------------+----------------+-----------------
 diagnosis |         899050 |         899050 | PASSED
 edstays   |         425087 |         425087 | PASSED
 medrecon  |        2987342 |        2987342 | PASSED
 pyxis     |        1586053 |        1586053 | PASSED
 triage    |         425087 |         425087 | PASSED
 vitalsign |        1564610 |        1564610 | PASSED
(6 rows)
```

```
2026-03-12 21:01:37 [INFO    ] Helper functions installed successfully.
2026-03-12 21:01:37 [INFO    ] ========================================
====================
2026-03-12 21:01:37 [INFO    ] STEP 17: Building MIMIC-IV derived conce
pts (postgres-make-concepts.sql)
2026-03-12 21:01:37 [INFO    ] ========================================
====================
```

```
SET
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
```

```
2026-03-12 21:01:37 [INFO    ] Building all concepts - this may take 1
5-60 minutes ...
2026-03-12 21:56:51 [INFO    ] Concepts build output:

===
Beginning to create materialized views for MIMIC database.
Any notices of the form  "NOTICE: materialized view "XXXXXX" does not e
xist" can be ignored.
The scripts drop views before creating them, and these notices indicate
nothing existed prior to creating the view.
===

SET
DROP TABLE
SELECT 94458
DROP TABLE
SELECT 10485609
DROP TABLE
SELECT 401850
DROP TABLE
SELECT 4127634
DROP TABLE
SELECT 4127634
DROP TABLE
SELECT 546028
DROP TABLE
SELECT 94458
DROP TABLE
SELECT 697418
```

```
DROP TABLE
SELECT 4154226
DROP TABLE
SELECT 380131
DROP TABLE
SELECT 4976408
DROP TABLE
SELECT 1991167
DROP TABLE
SELECT 4377900
DROP TABLE
SELECT 546028
DROP TABLE
SELECT 2187060
DROP TABLE
SELECT 2217787
DROP TABLE
SELECT 43342
DROP TABLE
SELECT 243283
DROP TABLE
SELECT 174269
DROP TABLE
SELECT 794232
DROP TABLE
SELECT 7887354
DROP TABLE
SELECT 4126485
DROP TABLE
SELECT 1377514
DROP TABLE
SELECT 13519533
DROP TABLE
SELECT 546028
DROP TABLE
SELECT 135153
DROP TABLE
SELECT 949901
DROP TABLE
SELECT 10264
DROP TABLE
SELECT 18085
DROP TABLE
SELECT 31495
DROP TABLE
SELECT 10668
DROP TABLE
SELECT 19430
DROP TABLE
SELECT 459800
DROP TABLE
SELECT 293253
```

```
DROP TABLE
SELECT 209376
DROP TABLE
SELECT 37163
DROP TABLE
SELECT 475214
DROP TABLE
SELECT 108165
DROP TABLE
SELECT 4098630
DROP TABLE
SELECT 144813
DROP TABLE
SELECT 94458
DROP TABLE
SELECT 94458
DROP TABLE
SELECT 94458
DROP TABLE
SELECT 94458
DROP TABLE
SELECT 94458
DROP TABLE
SELECT 94458
DROP TABLE
SELECT 94458
DROP TABLE
SELECT 94458
DROP TABLE
SELECT 94458
DROP TABLE
SELECT 94458
DROP TABLE
SELECT 811585
DROP TABLE
SELECT 94458
DROP TABLE
SELECT 94458
DROP TABLE
SELECT 94458
DROP TABLE
SELECT 94458
DROP TABLE
SELECT 94458
DROP TABLE
SELECT 94458
DROP TABLE
SELECT 94458
DROP TABLE
SELECT 8219121
DROP TABLE
SELECT 949901
DROP TABLE
SELECT 5099899
DROP TABLE
SELECT 94458
```

```
DROP TABLE
SELECT 41295
DROP TABLE
SELECT 839543
DROP TABLE
SELECT 783613

2026-03-12 21:56:51 [WARNING ] Concepts build stderr:
psql:demographics/icustay_times.sql:2: NOTICE:  table "icustay_times" d
oes not exist, skipping
psql:demographics/icustay_hourly.sql:2: NOTICE:  table "icustay_hourly"
does not exist, skipping
psql:demographics/weight_durations.sql:2: NOTICE:  table "weight_durati
ons" does not exist, skipping
psql:measurement/urine_output.sql:2: NOTICE:  table "urine_output" does
not exist, skipping
psql:organfailure/kdigo_uo.sql:2: NOTICE:  table "kdigo_uo" does not ex
ist, skipping
psql:demographics/age.sql:2: NOTICE:  table "age" does not exist, skipp
ing
psql:demographics/icustay_detail.sql:2: NOTICE:  table "icustay_detail"
does not exist, skipping
psql:measurement/bg.sql:2: NOTICE:  table "bg" does not exist, skipping
psql:measurement/blood_differential.sql:2: NOTICE:  table "blood_differ
ential" does not exist, skipping
psql:measurement/cardiac_marker.sql:2: NOTICE:  table "cardiac_marker"
does not exist, skipping
psql:measurement/chemistry.sql:2: NOTICE:  table "chemistry" does not e
xist, skipping
psql:measurement/coagulation.sql:2: NOTICE:  table "coagulation" does n
ot exist, skipping
psql:measurement/complete_blood_count.sql:2: NOTICE:  table "complete_b
lood_count" does not exist, skipping
psql:measurement/creatinine_baseline.sql:2: NOTICE:  table "creatinine_
baseline" does not exist, skipping
psql:measurement/enzyme.sql:2: NOTICE:  table "enzyme" does not exist,
skipping
psql:measurement/gcs.sql:2: NOTICE:  table "gcs" does not exist, skippi
ng
psql:measurement/height.sql:2: NOTICE:  table "height" does not exist,
skipping
psql:measurement/icp.sql:2: NOTICE:  table "icp" does not exist, skippi
ng
psql:measurement/inflammation.sql:2: NOTICE:  table "inflammation" does
not exist, skipping
psql:measurement/oxygen_delivery.sql:2: NOTICE:  table "oxygen_deliver
y" does not exist, skipping
psql:measurement/rhythm.sql:2: NOTICE:  table "rhythm" does not exist,
skipping
psql:measurement/urine_output_rate.sql:2: NOTICE:  table "urine_output_
rate" does not exist, skipping
psql:measurement/ventilator_setting.sql:2: NOTICE:  table "ventilator_s
```

```
etting" does not exist, skipping
psql:measurement/vitalsign.sql:2: NOTICE:  table "vitalsign" does not e
xist, skipping
psql:comorbidity/charlson.sql:2: NOTICE:  table "charlson" does not exi
st, skipping
psql:medication/acei.sql:2: NOTICE:  table "acei" does not exist, skipp
ing
psql:medication/antibiotic.sql:2: NOTICE:  table "antibiotic" does not
exist, skipping
psql:medication/dobutamine.sql:2: NOTICE:  table "dobutamine" does not
exist, skipping
psql:medication/dopamine.sql:2: NOTICE:  table "dopamine" does not exis
t, skipping
psql:medication/epinephrine.sql:2: NOTICE:  table "epinephrine" does no
t exist, skipping
psql:medication/milrinone.sql:2: NOTICE:  table "milrinone" does not ex
ist, skipping
psql:medication/neuroblock.sql:2: NOTICE:  table "neuroblock" does not
exist, skipping
psql:medication/norepinephrine.sql:2: NOTICE:  table "norepinephrine" d
oes not exist, skipping
psql:medication/nsaid.sql:2: NOTICE:  table "nsaid" does not exist, ski
pping
psql:medication/phenylephrine.sql:2: NOTICE:  table "phenylephrine" doe
s not exist, skipping
psql:medication/vasopressin.sql:2: NOTICE:  table "vasopressin" does no
t exist, skipping
psql:treatment/crrt.sql:2: NOTICE:  table "crrt" does not exist, skippi
ng
psql:treatment/invasive_line.sql:2: NOTICE:  table "invasive_line" does
not exist, skipping
psql:treatment/rrt.sql:2: NOTICE:  table "rrt" does not exist, skipping
psql:treatment/ventilation.sql:2: NOTICE:  table "ventilation" does not
exist, skipping
psql:firstday/first_day_bg.sql:2: NOTICE:  table "first_day_bg" does no
t exist, skipping
psql:firstday/first_day_bg_art.sql:2: NOTICE:  table "first_day_bg_art"
does not exist, skipping
psql:firstday/first_day_gcs.sql:2: NOTICE:  table "first_day_gcs" does
not exist, skipping
psql:firstday/first_day_height.sql:2: NOTICE:  table "first_day_height"
does not exist, skipping
psql:firstday/first_day_lab.sql:2: NOTICE:  table "first_day_lab" does
not exist, skipping
psql:firstday/first_day_rrt.sql:2: NOTICE:  table "first_day_rrt" does
not exist, skipping
psql:firstday/first_day_urine_output.sql:2: NOTICE:  table "first_day_u
rine_output" does not exist, skipping
psql:firstday/first_day_vitalsign.sql:2: NOTICE:  table "first_day_vita
lsign" does not exist, skipping
psql:firstday/first_day_weight.sql:2: NOTICE:  table "first_day_weight"
does not exist, skipping
```

```
psql:organfailure/kdigo_creatinine.sql:2: NOTICE:  table "kdigo_creatin
ine" does not exist, skipping
psql:organfailure/meld.sql:2: NOTICE:  table "meld" does not exist, ski
pping
psql:score/apsiii.sql:2: NOTICE:  table "apsiii" does not exist, skippi
ng
psql:score/lods.sql:2: NOTICE:  table "lods" does not exist, skipping
psql:score/oasis.sql:2: NOTICE:  table "oasis" does not exist, skipping
psql:score/sapsii.sql:2: NOTICE:  table "sapsii" does not exist, skippi
ng
psql:score/sirs.sql:2: NOTICE:  table "sirs" does not exist, skipping
psql:score/sofa.sql:2: NOTICE:  table "sofa" does not exist, skipping
psql:sepsis/suspicion_of_infection.sql:2: NOTICE:  table "suspicion_of_
infection" does not exist, skipping
psql:organfailure/kdigo_stages.sql:2: NOTICE:  table "kdigo_stages" doe
s not exist, skipping
psql:firstday/first_day_sofa.sql:2: NOTICE:  table "first_day_sofa" doe
s not exist, skipping
psql:sepsis/sepsis3.sql:2: NOTICE:  table "sepsis3" does not exist, ski
pping
psql:medication/vasoactive_agent.sql:2: NOTICE:  table "vasoactive_agen
t" does not exist, skipping
psql:medication/norepinephrine_equivalent_dose.sql:2: NOTICE:  table "n
orepinephrine_equivalent_dose" does not exist, skipping

2026-03-12 21:56:51 [INFO    ] All derived concepts built successfully.
==============================================================
Pipeline complete. MIMIC-IV, MIMIC-IV-ED, and derived concepts are read
y.
Connect via: psql -h localhost -p 5432 -U mimicuser -d mimiciv
Web UI: http://localhost:28080  (if Adminer container is running)
==============================================================
```